

Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 6: **Input Validation and Security**

FormRequest, Common Input-Based Attacks, and Totals Recomputed on the Server

What You'll Learn

So far, every form you've built has trusted whatever its user sends without question. This meeting changes that:

1. Three ways malicious input attacks a web app (**XSS**, **SQL Injection**, **CSRF**), and why the app you've already built is safe from most of them without you realizing it
2. How to write a **FormRequest** to validate input before the controller ever touches it
3. Why values like a transaction total must be recomputed on the server, never trusted directly from client input
4. How to show error messages and a **flash message** in Blade so users know why their input was rejected

This slide deck covers concepts. Applying all of this to Simple POS happens in the practicum jobsheet.

Part 1

Input Security Basics

Why User Input Can't Be Trusted

Every text box, every URL parameter, every form field is a door anyone can walk through, including someone deliberately trying to break or steal data. The three most common attacks through that door: **XSS**, **SQL Injection**, and **CSRF**.

Good news: you've already been using some of these defenses since Meeting 3 and 5, without knowing exactly why they mattered. The next three slides explain why.

XSS: Cross-Site Scripting

XSS (Cross-Site Scripting): an attack that injects JavaScript into a page through user input, so that code runs in another visitor's browser once the page is rendered.

Imagine an article's comment field accepts `<script>` input and displays it as-is to the next visitor, that script now runs in their browser, able to steal cookies or an active session.

```
{{-- Blade escapes automatically, safe --}}  
<p>{{ $comment->body }}</p>  
  
{{-- Blade does NOT escape, vulnerable to XSS --}}  
<p>{!! $comment->body !!}</p>
```

You've been avoiding this since Meeting 3: `{{ }}` has always been the default choice in every view you've built.

SQL Injection

SQL Injection: an attack that injects a piece of SQL command through input, exploiting a query built by concatenating raw strings instead of using separate parameters.

```
// Vulnerable: input concatenated directly into the SQL string
DB::select("SELECT * FROM users WHERE email = '$email'");

// Safe: Eloquent always uses bound parameters
User::where('email', $email)->first();
```

If `$email` contains `' OR '1'='1'`, the vulnerable version above could return every row, not just one. Every Eloquent query you've written since Meeting 5 is automatically safe from this, because Eloquent always sends values through bound parameters, never by concatenating strings.

CSRF: Cross-Site Request Forgery

CSRF (Cross-Site Request Forgery): an attack that tricks a victim's browser into sending a request to another application using the victim's still-active login session, without their knowledge.

Imagine you're logged into a cashier app, then you open another site that quietly sends a `POST /pos` form to that app through your own browser, using your still-active login session.

```
<form method="POST" action="/pos">
  @csrf
  <!-- ...other inputs... -->
</form>
```

The `@csrf` token you've always written in every Blade form since Meeting 3 is the defense: Laravel rejects any POST request that doesn't carry that token.

Full list of common vulnerabilities: OWASP Top 10, owasp.org/www-project-top-ten

Part 2

FormRequest and Validation Rules

FormRequest: A Guard Before the Controller

FormRequest: a Laravel class that wraps a single request's validation and authorization logic, separate from the controller, so the controller only ever receives data that's already been declared valid.

```
php artisan make:request StoreArticleRequest
```

The moment a controller accepts a parameter typed as `StoreArticleRequest`, Laravel automatically runs its validation rules first. If anything fails, the user is sent straight back to the form with the error messages, the code inside the controller never even runs.

Writing Validation Rules

```
class StoreArticleRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255'],
            'author_id' => ['required', 'exists:authors,id'],
            'body' => ['required', 'string'],
        ];
    }
}
```

`authorize()` decides who's allowed to use this form, `true` means anyone who got this far is allowed to submit it. `exists:authors,id` runs a database query, rejecting an `author_id` that doesn't actually exist.

Validating Arrays: Wildcard Notation

When a single request carries a list of items, like a shopping cart, the `*` wildcard validates every row without a manual loop:

```
public function rules(): array
{
    return [
        'items' => ['required', 'array', 'min:1'],
        'items.*.product_id' => ['required', 'exists:products,id'],
        'items.*.qty' => ['required', 'integer', 'min:1'],
    ];
}
```

Every element in `items` is checked one by one using the same rules. When a row is invalid, the whole request is rejected, and the error message points at exactly the offending row, e.g. `items.2.qty`.

Part 3

Totals Recomputed on the Server

Why Values From the Client Can't Be Trusted

Any value sent from the client, even if it looks correctly calculated on screen, can be changed before it actually reaches the server. A total field sent through an HTML form is no exception: it's just plain text inside the request, and nothing stops someone from changing it before that request is ever sent.

The Alpine.js cart you built (Meeting 3) computes a subtotal in the browser purely for display, not to be trusted as the final number.

Recomputing on the Server

```
$total = 0;

foreach ($validated['items'] as $item) {
    $product = Product::findOrFail($item['product_id']);
    $subtotal = $product->price * $item['qty'];
    $total += $subtotal;

    // save the transaction detail row with $subtotal here
}

$transaction->update(['total' => $total]);
```

Prices are re-fetched from the database, not from input, then multiplied by the requested quantity. The client can send whatever it wants in its `total` field, the server always computes its own and ignores it entirely.

Part 4

Error Messages and Flash Messages

Showing Error Messages in Blade

```
<input type="text" name="title" value="{{ old('title') }}">
@error('title')
    <p class="text-sm text-red-600">{{ $message }}</p>
@enderror
```

`@error('title')` automatically shows that message when the `title` field fails validation, and shows nothing when it passes. `old('title')` refills the input with what the user submitted last, so the form isn't empty again after a failed validation.

Flash Message

Flash Message: a short message stored in the session for only the next request, used to show a success or failure notification after a redirect.

```
return redirect()  
    ->route('transactions.show', $transaction)  
    ->with('success', 'Transaksi berhasil disimpan.');
```

```
@if (session('success'))  
    <div class="bg-green-50 text-green-700 p-3 rounded-md mb-4">  
        {{ session('success') }}  
    </div>  
@endif
```

That message shows up once after the redirect, then disappears on its own if the page reloads, because it's only ever stored for one request ahead.

Applying This to Simple POS

You'll apply the FormRequest, validation, and server-side total concepts directly to Simple POS in the practicum jobsheet: adding `$fillable` to the models (the mass assignment concept from Meeting 5), writing `StoreProductRequest` and `StoreTransactionRequest` for the models you've already built since Meeting 4 and 5, then making sure the transaction total is always recomputed from database prices, never from cashier input.

Full code: github.com/se-polinema/simple-pos-ch06

Summary

- XSS, SQL Injection, and CSRF are three common input-based attacks; Blade (`{{ }}`), Eloquent (bound parameters), and `@csrf` have already been default defenses since earlier meetings
- FormRequest wraps validation rules separately from the controller, running automatically before the controller ever executes, including array validation through `items.*.field` notation
- Values like a transaction total must be recomputed on the server from database data, never trusted directly from client input
- `@error()` shows a per-field error message, `old()` refills failed input, and a flash message via `session()` shows a one-time notification after a redirect

References & Discussion

Official Laravel documentation (Validation, FormRequest), OWASP Top 10

Full code: `github.com/se-polinema/simple-pos`

Next meeting: Authentication & Authorization